

Package ‘tsinteractions’

September 6, 2023

Title Robust Dynamic Inferences from Single-Equation Time Series
Models with Interactions

Version 0.0.1

Maintainer Soren Jordan <sorenjordanpols@gmail.com>

Description While scholars are increasingly interested in estimating relationships between dynamic variables that have effects conditional on the values of each other (interactive effects), the literature is largely silent on two key points. The first is how to calculate traditional measures of dynamic effects, like long- and short-run effects, in the presence of dynamic conditional relationships. The second is how to visualize those effects in substantively useful ways for readers. This package allows for the graphical exploration of conditional effects from interactions in time series.

Imports car, ggplot2, colorspace

Suggests knitr, rmarkdown, testthat (>= 3.0.0)

Depends R (>= 3.5.0)

License GPL (>=2)

Encoding UTF-8

LazyData true

BuildManual yes

RoxygenNote 7.2.3

VignetteBuilder knitr

Config/testthat/edition 3

NeedsCompilation no

Author Soren Jordan [aut, cre, cph],
Garrett N. Vande Kamp [aut]

R topics documented:

approval	2
toy.ts.interaction.data	3
ts_interact_plot	4

Index	7
--------------	----------

approval

*Data on US Presidential Approval***Description**

A dataset from: Cavari, Amnon. 2019. "Evaluating the President on Your Priorities: Issue Priorities, Policy Performance, and Presidential Approval, 1981–2016." *Presidential Studies Quarterly* 49(4): 798-826.

Usage

```
data(approval)
```

Format

A data frame with 140 rows and 14 variables:

APPROVE Presidential approval

APPROVE_ECONOMY Presidential approval: economy

APPROVE_FOREIGN Presidential approval: foreign affairs

MIP_MACROECONOMICS Salience (Most Important Problem): economy

MIP_FOREIGN Salience (Most Important Problem): foreign affairs

PARTY_IN Macropartisanship (in-party)

PARTY_OUT Macropartisanship (out-party)

PRESIDENT Numeric indicator for president

DIVIDEDGOV Dummy variable for divided government

ELECTION Dummy variable for election years

HONEYMOON Dummy variable for honeymoon period

UMCSENT Consumer sentiment

UNRATE Unemployment rate

APPROVE_L1 Lagged presidential approval

Source

[doi:10.1111/psq.12594](https://doi.org/10.1111/psq.12594)

`toy.ts.interaction.data`*Simulated interactive time series data*

Description

A simulated, well-behaved dataset of interactive time series data

Usage

```
data(toy.ts.interaction.data)
```

Format

A data frame with 50 rows and 23 variables:

time Indicator for time period

x Contemporaneous x

l.1.x First lag of x

l.2.x Second lag of x

l.3.x Third lag of x

l.4.x Fourth lag of x

l.5.x Fifth lag of x

z Contemporaneous z

l.1.z First lag of z

l.2.z Second lag of z

l.3.z Third lag of z

l.4.z Fourth lag of z

l.5.z Fifth lag of z

y Contemporaneous y

l.1.y First lag of y

l.2.y Second lag of y

l.3.y Third lag of y

l.4.y Fourth lag of y

l.5.y Fifth lag of y

x.z Interaction of contemporaneous x and z

x.l.1.z Interaction of contemporaneous x and lagged z

z.l.1.x Interaction of lagged x and contemporaneous z

l.1.x.l.1.z Interaction of lagged x and lagged z

ts_interact_plot	<i>Plot the interaction in a single-equation time series model estimated via lm</i>
------------------	---

Description

Plot the interaction in a single-equation time series model estimated via lm

Usage

```
ts_interact_plot(
  model = NULL,
  x.vrbl = NULL,
  z.vrbl = NULL,
  y.vrbl = NULL,
  x.z.vrbl = NULL,
  effect.type = "x.me",
  dM.level = 0.95,
  return.data = FALSE,
  return.plot = TRUE,
  bw.lines = FALSE,
  heatmap.colors = "Blue-Red 3",
  line.colors = NULL,
  z.vals = NULL,
  z.limits = NULL,
  s.limit = 20,
  z.label.rounding = 0,
  s.plot.vals = c(0, "LRM"),
  ...
)
```

Arguments

model	lm model that contains the time series interaction
x.vrbl	string with the “main” x variable: usually presented along x-axis. If lags are included, a string of the lags in order (i.e. 0, 1, 2)
z.vrbl	string with the “moderating” z variable: line type, y-axis, or other. If lags are included, a string of the lags in order (i.e. 0, 1, 2)
y.vrbl	string with the dependent variable from the model
x.z.vrbl	string with the interaction variable
effect.type	one of x.me (marginal effect of x), x.cme (cumulative marginal effect of x), xz.me.zlines (marginal effects, with the line types by values of the moderating variable), xz.me.heatmap.onlysig (heatmap of the marginal effects, but colored white if the effect does not meet the dM.level), xz.me.heatmap (heatmap of the marginal effects, as estimated), xz.cme.zlines (cumulative marginal effects, with the line types by values of the moderating variable), xz.cme.slines (cumulative marginal effects, with the line types by values of the period since the shock), xz.cme.heatmap.onlysig (heatmap of the cumulative marginal effects, but colored white if the effect does not meet the dM.level), or xz.cme.heatmap (heatmap of the marginal effects, as estimated)

dm.level	the significance level of the (cumulative) marginal effects: set to deltaMethod
return.data	return the estimated effects dataframe in the created object (as opposed to just creating the plot)
return.plot	return the plot to the console (as opposed to just creating the data)
bw.lines	should the line colors be black and white or full color? The option is labeled “lines,” since a black and white heatmap would be dubiously useful
heatmap.colors	what color scale would you like for the heatmap? The default is “Blue-Red 3”
line.colors	what color lines would you like for line type plots? If bw.lines is not TRUE, this defaults to the color-safe Okabe-Ito colors
z.vals	values for the moderating variable, if a line type plot is to be drawn
z.limits	what is the range of values of the moderating variable to plot, if a heatmap is to be drawn
s.limit	number of time periods from a shock to calculate
z.label.rounding	number of digits to round to for the z labels in the legend (if those values are automatically calculated)
s.plot.vals	values for the time since the shock, if a line type plot is to be drawn
...	other arguments to be passed to the call to plot

Author(s)

Soren Jordan and Garrett N. Vande Kamp

Examples

```
# Using Cavari's approval data in dynamac
# Cavari's original model: APPROVE ~ APPROVE_ECONOMY + APPROVE_FOREIGN + MIP_MACROECONOMICS +
#   MIP_FOREIGN + APPROVE_ECONOMY*MIP_MACROECONOMICS + APPROVE_FOREIGN*MIP_FOREIGN +
#   APPROVE_L1 + PARTY_IN + PARTY_OUT + UNRATE +
#   DIVIDEDGOV + ELECTION + HONEYMOON + as.factor(PRESIDENT)

approval$ECONAPP_ECONMIP <- approval$APPROVE_ECONOMY*approval$MIP_MACROECONOMICS
approval$FPAPP_ECONFP <- approval$APPROVE_FOREIGN*approval$MIP_FOREIGN
approval$PRESIDENT.num <- cbind(approval$PRESIDENT)

# As of 0.0.1, only works with lm()
cavari.model <- lm(APPROVE ~ APPROVE_ECONOMY + APPROVE_FOREIGN + MIP_MACROECONOMICS +
  MIP_FOREIGN + ECONAPP_ECONMIP + FPAPP_ECONFP +
  APPROVE_L1 + PARTY_IN + PARTY_OUT + UNRATE +
  DIVIDEDGOV + ELECTION + HONEYMOON + as.factor(PRESIDENT.num), data = approval)

# If model included lags of x.vrbl, we would add them to c()
# First: cumulative marginal effect of X
ts_interact_plot(model = cavari.model, x.vrbl = c("APPROVE_ECONOMY"), y.vrbl = "APPROVE_L1",
  effect.type = "x.cme")

# Now: marginal effect of X at different levels of Z
ts_interact_plot(model = cavari.model, x.vrbl = c("APPROVE_ECONOMY"), y.vrbl = "APPROVE_L1",
  z.vrbl = "MIP_MACROECONOMICS", x.z.vrbl = "ECONAPP_ECONMIP",
  effect.type = "xz.me.zlines")

# Use well-behaved simulated data (included) for even more examples
```

```
model.toydata <- lm(y ~ l.1.y + x + l.1.x + z + l.1.z +  
x.z + l.1.x.l.1.z, data = toy.ts.interaction.data)  
  
# Cumulative marginal effect of x. You can store the data to draw your own plot,  
# if you prefer  
test.x.cme <- ts_interact_plot(model = model.toydata,  
x.vrbl = c("x", "l.1.x"), y.vrbl = "l.1.y",  
effect.type = "x.cme", return.data = TRUE, s.limit = 20)  
test.x.cme$plot  
  
# Marginal effect of z  
ts_interact_plot(model = model.toydata, x.vrbl = c("x", "l.1.x"),  
y.vrbl = "l.1.y", z.vrbl = c("z", "l.1.z"),  
x.z.vrbl = c("x.z", "l.1.x.l.1.z"),  
z.vals = -2:2,  
effect.type = "xz.me.zlines", s.limit = 20)
```

Index

- * **datasets**
 - approval, [2](#)
 - toy.ts.interaction.data, [3](#)
- * **interaction**
 - ts_interact_plot, [4](#)
- * **plot**
 - ts_interact_plot, [4](#)

approval, [2](#)

toy.ts.interaction.data, [3](#)

ts_interact_plot, [4](#)